

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

FALL SEMESTER 2026-2027

BACSE202 - DATABASE SYSTEMS

DIGITAL ASSIGNMENT - TECHNICAL REPORT

Stock Anomaly Detection and Real-Time Monitoring System

Faculty Name:	Dr. S. Geetha
Slot:	D1 + TD1
Class No:	CH202670101707
Team Members & Reg Nos:	HARISH R – 25BCE1060 SUBRAHMANIAN RAMAKRISHNAN – 25BCE1094 PRITHYU RAM B – 25BCE1126
GitHub Repository Link:	https://github.com/harishramaswamy28-oss/StockAnamolyDetection.git
YouTube Demo Video Link:	https://youtu.be/4V1y-uxerMg?si=QYuTx-DsTZSvm062

1. Abstract & Keywords

Stock prices move constantly, and fixed alert thresholds cannot tell whether a move is unusual for one particular stock. This project presents a Stock Anomaly Detection and Real-Time Monitoring System that ingests daily and minute OHLCV bars from Alpaca Markets, learns each security's normal behaviour from rolling return and volume distributions, and flags deviations using adaptive Tukey (IQR) fences. Detected anomalies are stored with full traceability to the analysis run and price bar that produced them, and users receive deduplicated notifications for anomalies on their watchlists and for their own price alerts. The system uses PostgreSQL with the TimescaleDB extension, a Python FastAPI backend connected through psycopg3, and a React dashboard. A normalised 15-table schema with composite keys, weak entities, CHECK constraints and partial unique indexes enforces integrity, and a pytest suite validates detection, alerting and API logic.

Keywords: Anomaly Detection, Time-Series Database, TimescaleDB, PostgreSQL, Interquartile Range (IQR), Real-Time Monitoring

2. Introduction

2.1 Problem Statement

Individual investors and analysts follow many stocks, but a price or volume change that is normal for one stock can be extraordinary for another, and what counts as "normal" changes over time. Most retail tools only let a user set a fixed price target, so users either miss genuinely unusual activity or are flooded with alerts that carry no explanation.

Target users: retail investors, students and analysts who want to be told when a stock behaves abnormally and to watch a personal list of securities.

Current pain points:

- Fixed thresholds ignore each stock's own volatility and volume pattern.
- Historical, daily and intraday data are easily mixed, which distorts baselines.
- Users receive repeated notifications for the same event.
- There is no audit trail showing which method and which bound flagged an observation.
- Alerts and watchlists belong to individual users and must stay private and consistent.

Why a database-backed solution: the system stores large, ordered market time series (daily and minute bars) and needs referential integrity between securities, bars, analysis runs, anomalies, users, watchlists, alerts and notifications. A relational DBMS with a time-series extension provides composite keys, foreign keys, CHECK and UNIQUE constraints, indexed range queries, atomic transactions and concurrency-safe de-duplication, all of which would have to be re-implemented by hand with flat files.

2.2 Objectives & Scope

The system has the following objectives:

1. Collect and store historical stock market data (daily and minute OHLCV) from Alpaca Markets.
2. Establish each stock's normal behaviour from rolling return and volume distributions and a rolling average volume.
3. Receive intraday minute bars through the Alpaca WebSocket and process them as they arrive.
4. Compare every new observation with its rolling baseline, with the current bar excluded from its own baseline.
5. Detect unusual price and volume movements using data-derived interquartile (Tukey) fences whose numeric bounds change with recent observations.
6. Store each anomaly with traceability to the analysis run that produced it and to the exact price bar it evaluates.
7. Let users define price alerts that work independently of statistical anomalies.
8. Notify users of new anomalies on their watchlists and of triggered alerts, without duplicates or historical noise.
9. Expose the functionality through a REST API (FastAPI) with password-based sign-in.
10. Provide a React dashboard for stocks, charts, anomalies, watchlists, alerts, notifications and preferences.

Scope.

In scope: four configured symbols (AAPL, MSFT, TSLA, NVDA) on the Alpaca IEX feed; daily and minute bars; rolling-IQR price and volume anomaly detection; in-app notifications; user accounts, watchlists, alerts and preferences; a web dashboard.

Out of scope: trading or order placement, a consolidated all-exchange (SIP) feed, e-mail/SMS delivery (the `email` preference value is stored only), time-of-day-normalised intraday baselines, machine-learning models, and mobile applications.

3. Existing Systems Analysis

Existing market-monitoring products were reviewed to see how they handle alerts and abnormal behaviour. The descriptions below are general and are based on each product's publicly documented purpose.

System / Reference	What it offers	Gaps this project addresses
TradingView https://www.tradingview.com	Charting, watchlists and user-configured price or indicator alerts.	Alerts are rules the user must define; thresholds are not learned per stock automatically, and there is no user-visible relational record of detected anomalies and the analysis run behind them.
Yahoo Finance https://finance.yahoo.com	Quotes, charts, watchlists and price notifications for retail users.	Notifications rely on fixed price or percentage triggers, with no volume-distribution anomaly logic or explanation of why a move is unusual.

System / Reference	What it offers	Gaps this project addresses
Bloomberg Terminal https://www.bloomberg.com/professional/products/bloomberg-terminal/	Professional market data, analytics and alerting.	Costly subscription and a closed system; its internals cannot be inspected, extended or reused for coursework or personal projects.
Alpaca Markets API https://docs.alpaca.markets	Brokerage and market-data API with REST and WebSocket access.	Supplies raw bars only, with no anomaly detection, user management, alert storage or notifications. It is used here as the data source.
Anomaly-detection research [7], [8]	General-purpose statistical and machine-learning detectors such as Tukey fences and Isolation Forest.	Models can be hard to explain per alert. This project applies the transparent Tukey rule and stores the exact bound used with every anomaly.

Gaps overcome by this system:

- Per-stock, data-derived thresholds that are recalculated as new observations arrive (no preset numeric cut-offs).
- Every anomaly is traceable to an `analysis_run` and a specific `price_bar`, and stores the bound that triggered it.
- Anomaly detection and user price alerts are kept independent in both the schema and the code.
- Notifications are de-duplicated by database constraints, and only users who watch the affected security are notified.
- The whole design is open, normalised and inspectable, suiting a database-systems study.

4. Requirements & System Architecture

4.1 Functional & Non-Functional Requirements

Functional requirements

ID	Requirement	Implemented in
FR1	Ingest daily and minute OHLCV bars and company, sector and exchange metadata from Alpaca.	`ingestion/price_ingestion.py`, `metadata_ingestion.py`
FR2	Detect price and volume anomalies on daily bars using rolling IQR fences.	`analysis/run_analysis.py`, `indicators.py`
FR3	Process live minute bars from the Alpaca WebSocket and detect intraday anomalies.	`realtime/stream_processor.py`, `analysis/realtime_detector.py`, `pipeline.py`
FR4	Store anomalies with traceability to the analysis run and the price bar.	`anomaly`, `analysis_run` tables
FR5	Register, sign in and sign out with password-based sessions.	`api/auth.py`, `/auth/*` routes
FR6	Create and delete watchlists; add and remove securities.	`/users/{id}/watchlists` routes
FR7	Create and delete price alerts (above or below a target) and evaluate them manually or automatically.	`alerts/alert_checker.py`, `/alerts/check`
FR8	Create de-duplicated in-app notifications for anomalies (watchers only) and triggered alerts; mark them as read.	`alerts/anomaly_notifier.py`, `/notifications` routes

ID	Requirement	Implemented in
FR9	View and update notification preferences.	`/users/{id}/preferences`
FR10	Visualise prices, volume, anomaly markers, current adaptive bounds and recent anomalies.	`dashboard/` (React, Chart.js)
FR11	Offer a guided, read-only query builder over the user's own records.	`api/user_query_builder.py`
FR12	Run background automation: data refresh, daily analysis, alert checks and stream reconnection.	`realtime/automation.py`

Non-functional requirements

Category	Requirement and how it is met
Data integrity	Primary, foreign and composite keys; NOT NULL, UNIQUE and CHECK constraints; composite foreign key from `anomaly` to `price_bar`; `chk_notification_source` allows exactly one notification source.
Security	Passwords hashed with scrypt and a random salt; session tokens stored only as SHA-256 hashes in `app_session` with 7-day expiry; HttpOnly, SameSite=Lax cookie; per-user ownership checks on every user route; parameterised SQL throughout; credentials kept in `.env` and never returned by the API.
Performance	TimescaleDB hypertable on `price_bar`; composite primary-key index plus `idx_price_bar_security_time`; indexes on `anomaly` and `notification`; baselines read only the last N bars; price queries capped at 5,000 rows; optional columnstore policy for chunks older than 14 days.
Reliability and concurrency	One automation owner elected with a PostgreSQL advisory lock; unique indexes with `ON CONFLICT DO NOTHING` make notification inserts safe under concurrent workers; the stream reconnects with capped back-off; transactions roll back on error.
Input validation	Pydantic field constraints, explicit checks and a consistent JSON error body with correct HTTP status codes (Section 7.1).
Explainability	The bound used is stored with each anomaly, `analysis_run.method` records the method version, and `/stocks/{symbol}/thresholds` shows the current limits and sample size.
Maintainability	Modular packages (`ingestion`, `analysis`, `alerts`, `realtime`, `api`, `dashboard`), central `config.py`, and a pytest suite with mocked database and Alpaca calls.
Usability	Single-page dashboard that refreshes every 15 seconds without a manual reload.

4.2 Proposed System Architecture

The system follows a three-layer architecture. The **presentation layer** is a React + Vite single-page dashboard that talks to the backend over HTTP/JSON using a session cookie. The **application layer** is a Python FastAPI service that contains the REST API and authentication, the real-time analysis pipeline, the alert checker and notifier, and background automation workers. The **data layer** is PostgreSQL with the TimescaleDB extension, reached through the psycopg3 driver. Alpaca Markets is the external data source.

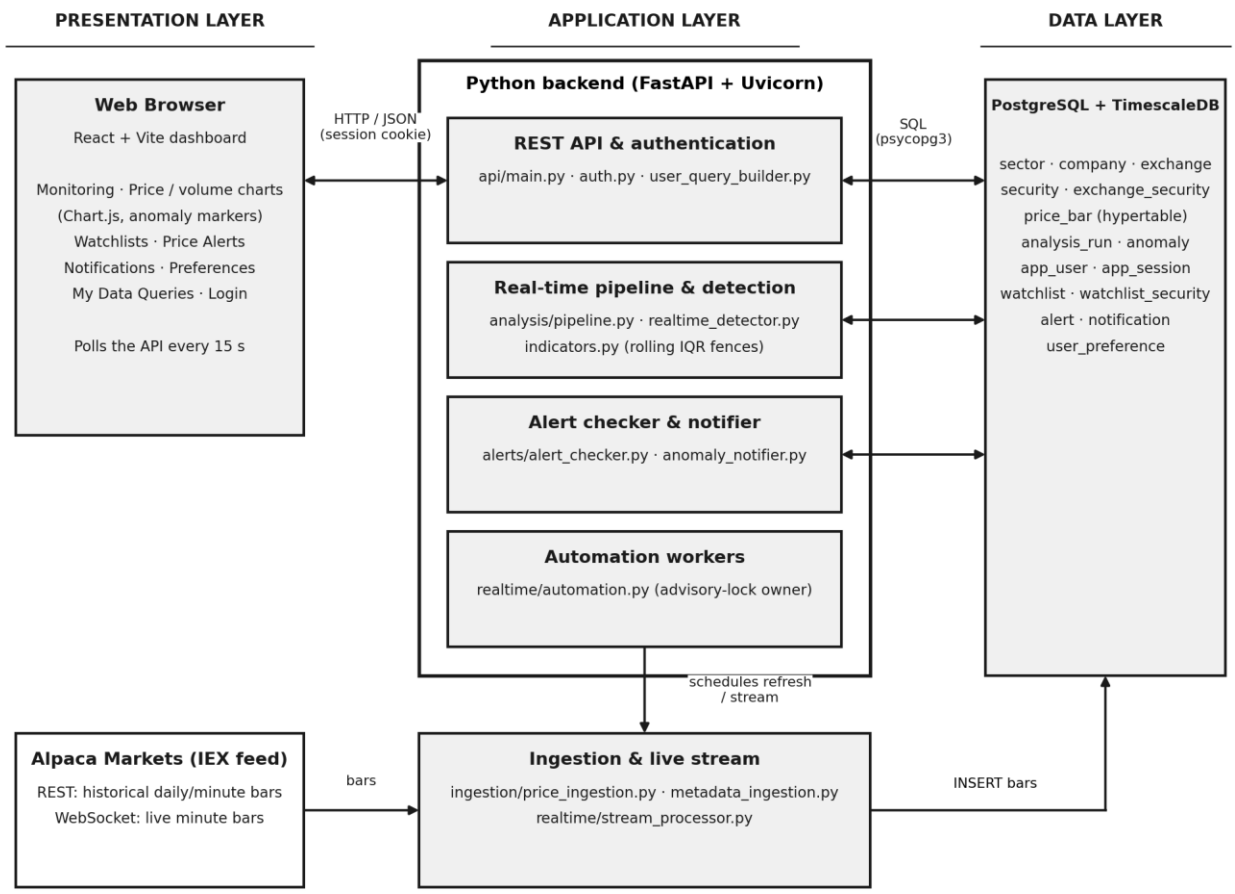


Figure 4.1: Proposed system architecture

End-to-end data flow:

- Metadata and historical daily and minute bars are fetched from Alpaca through REST and written to `security`, `company`, `sector`, `exchange` and `price_bar` (bars are upserted).
- Daily analysis computes rolling return and volume fences per security and records each run in `analysis_run` and each finding in `anomaly`.
- The stream processor receives live minute bars over an authenticated WebSocket and calls `analysis/pipeline.py`, which stores the bar, evaluates it against the prior-window baseline, stores any anomaly, notifies watching users and checks active alerts.
- The React dashboard calls the REST API to read prices, anomalies, thresholds, watchlists, alerts and notifications, and to create or change user data.
- Automation workers owned by one API process refresh data every six hours, re-run daily analysis, check alerts every minute and restart the stream if it drops.

5. Database Design & Modeling

5.1 Entity-Relationship (ER) Diagram

The ER diagram below uses Chen notation: rectangles are entities (double rectangles are weak entities), ellipses are attributes (underlined for primary keys, dashed underline for partial keys), and diamonds are relationships (double diamonds are identifying relationships). Cardinalities are marked on each link. The next page shows the diagram, followed by the entity and relationship summaries.

Entity	Type	Key attributes	Description
SECTOR	Strong	sector_id	Industry classification
COMPANY	Strong	company_id	Issuing company; belongs to one sector
EXCHANGE	Strong	exchange_id	Listing exchange (name, country, timezone)
SECURITY	Strong	security_id; symbol (candidate key)	Tradable instrument
PRICE_BAR	Weak	security_id + timestamp + timeframe	OHLCV observation (day or minute)
ANALYSIS_RUN	Strong	analysis_id	One traceable execution of a detection method
ANOMALY	Strong	anomaly_id	Detected deviation, tied to a run and a bar
USER (app_user)	Strong	user_id; email (candidate key)	Application user
WATCHLIST	Weak	user_id + watchlist_no	Named list of securities owned by a user
ALERT	Weak	user_id + alert_no	User-defined price alert
NOTIFICATION	Strong	notification_id	Message sourced from exactly one alert or anomaly
PREFERENCE (user_preference)	Weak (1:1)	user_id	Per-user notification preferences

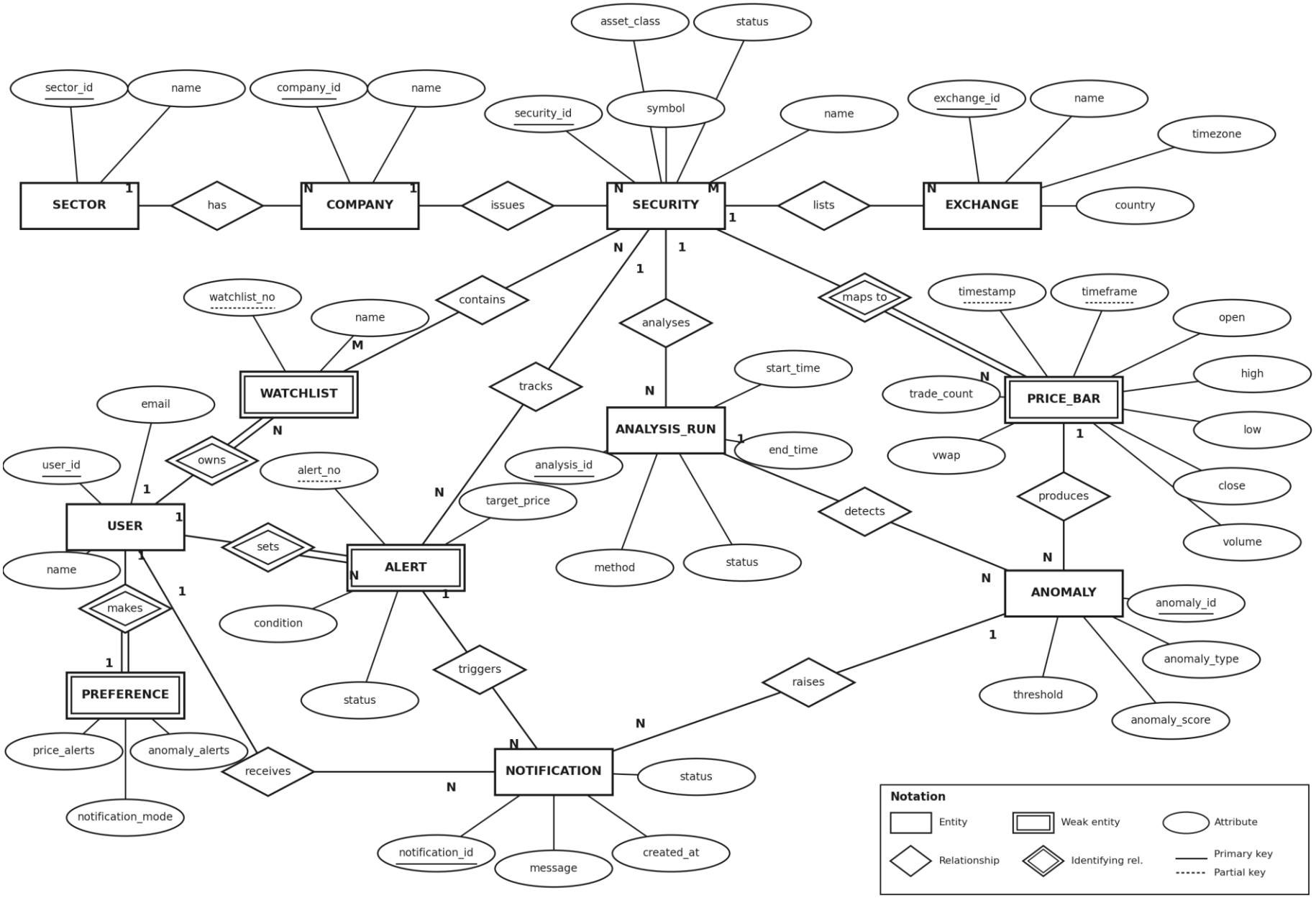


Figure 5.1: Entity-Relationship (ER) diagram of the Stock Anomaly Detection system

Relationships and cardinalities

Relationship	Entities	Cardinality	Implementation
has	SECTOR - COMPANY	1 : N	`company.sector_id` foreign key
issues	COMPANY - SECURITY	1 : N	`security.company_id` foreign key
lists	EXCHANGE - SECURITY	M : N	Associative table `exchange_security`
maps to (identifying)	SECURITY - PRICE_BAR	1 : N	`price_bar` key includes `security_id`
analyses	SECURITY - ANALYSIS_RUN	1 : N	`analysis_run.security_id` foreign key
produces	PRICE_BAR - ANOMALY	1 : N	Composite foreign key (`security_id, timestamp, timeframe`)
detects	ANALYSIS_RUN - ANOMALY	1 : N	`anomaly.analysis_id` foreign key
owns (identifying)	USER - WATCHLIST	1 : N	Key (`user_id, watchlist_no`)
contains	WATCHLIST - SECURITY	M : N	Associative table `watchlist_security`
sets (identifying)	USER - ALERT	1 : N	Key (`user_id, alert_no`)
tracks	ALERT - SECURITY	N : 1	`alert.security_id` foreign key
makes (identifying)	USER - PREFERENCE	1 : 1	`user_preference.user_id` is primary and foreign key
receives	USER - NOTIFICATION	1 : N	`notification.user_id` foreign key
triggers	ALERT - NOTIFICATION	1 : N	Foreign key (`user_id, alert_no`)
raises	ANOMALY - NOTIFICATION	1 : N	`notification.anomaly_id` foreign key

A notification has exactly one source, either an alert or an anomaly. This is enforced by the CHECK constraint `chk_notification_source` rather than shown as a separate relationship symbol. The authentication table `app_session` (`token_hash`, `user_id`, `expires_at`, `created_at`, added by `sql/05_user_authentication.sql`) is a technical table for sign-in sessions and is therefore omitted from the conceptual diagram; it appears in the relational schema below.

5.2 Relational Schema & Constraints

The ER diagram was converted using the standard mapping rules: each strong entity becomes a relation with its key; a weak entity takes the key of its owner plus its partial key; a 1:N relationship becomes a foreign key on the N side; an M:N relationship becomes an associative relation whose key combines the participating keys; and the 1:1 relationship is implemented with a primary key that is also a foreign key. Notation: **PK** primary key, **FK** foreign key, **UK** unique key.

Relation	Attributes	Keys and integrity constraints
sector	sector_id, name	PK(sector_id); name NOT NULL, UK
company	company_id, name, sector_id	PK(company_id); FK sector_id → sector; name, sector_id NOT NULL

Relation	Attributes	Keys and integrity constraints
exchange	exchange_id, name, country, timezone	PK(exchange_id); name NOT NULL, UK
security	security_id, symbol, name, asset_class, status, company_id	PK(security_id); UK(symbol); FK company_id → company; symbol, name, company_id NOT NULL
exchange_security	exchange_id, security_id	PK(exchange_id, security_id); FK exchange_id → exchange and security_id → security, both ON DELETE CASCADE
price_bar	security_id, timestamp, timeframe, open, high, low, close, volume, trade_count, vwap	PK(security_id, timestamp, timeframe); FK security_id → security (CASCADE); CHECK timeframe IN ('day','minute'); OHLCV and volume NOT NULL; hypertable on timestamp
analysis_run	analysis_id, security_id, start_time, end_time, method, status	PK(analysis_id); FK security_id → security (CASCADE); CHECK end_time >= start_time; all columns NOT NULL
anomaly	anomaly_id, security_id, timestamp, timeframe, analysis_id, anomaly_score, threshold, anomaly_type	PK(anomaly_id); FK(security_id, timestamp, timeframe) → price_bar (CASCADE); FK analysis_id → analysis_run (CASCADE); UK(analysis_id, security_id, timestamp, timeframe, anomaly_type); all columns NOT NULL
app_user	user_id, name, email, password_hash	PK(user_id); UK(email); name, email NOT NULL; password_hash nullable for legacy accounts
app_session	token_hash, user_id, expires_at, created_at	PK(token_hash) CHAR(64); FK user_id → app_user (CASCADE); created_at DEFAULT CURRENT_TIMESTAMP
watchlist	user_id, watchlist_no, name	PK(user_id, watchlist_no); FK user_id → app_user (CASCADE); name NOT NULL
watchlist_security	user_id, watchlist_no, security_id	PK(user_id, watchlist_no, security_id); FK(user_id, watchlist_no) → watchlist (CASCADE); FK security_id → security (CASCADE)
alert	user_id, alert_no, security_id, condition, target_price, status	PK(user_id, alert_no); FK user_id → app_user (CASCADE); FK security_id → security; CHECK condition IN ('above','below'); status DEFAULT 'active'
notification	notification_id, user_id, alert_no, anomaly_id, message, created_at, status	PK(notification_id); FK user_id → app_user; FK(user_id, alert_no) → alert; FK anomaly_id → anomaly; CHECK chk_notification_source; UK(user_id, anomaly_id); unique partial index on (user_id, alert_no) WHERE alert_no IS NOT NULL AND status='unread'; status DEFAULT 'unread'; created_at DEFAULT CURRENT_TIMESTAMP
user_preference	user_id, price_alerts, anomaly_alerts, notification_mode	PK and FK user_id → app_user (CASCADE); price_alerts, anomaly_alerts DEFAULT TRUE; notification_mode DEFAULT 'in_app' with CHECK IN ('in_app','email')

5.3 Functional Dependencies & Normalization (Up to 3NF)

Functional dependencies. The dependencies below hold in the schema (K denotes a candidate key).

Relation	Functional dependencies
sector	sector_id → name; name → sector_id
company	company_id → name, sector_id
exchange	exchange_id → name, country, timezone; name → exchange_id, country, timezone

Relation	Functional dependencies
security	security_id → symbol, name, asset_class, status, company_id; symbol → security_id
price_bar	(security_id, timestamp, timeframe) → open, high, low, close, volume, trade_count, vwap
analysis_run	analysis_id → security_id, start_time, end_time, method, status
anomaly	anomaly_id → security_id, timestamp, timeframe, analysis_id, anomaly_score, threshold, anomaly_type; (analysis_id, security_id, timestamp, timeframe, anomaly_type) → anomaly_id, anomaly_score, threshold
app_user	user_id → name, email, password_hash; email → user_id
app_session	token_hash → user_id, expires_at, created_at
watchlist	(user_id, watchlist_no) → name
alert	(user_id, alert_no) → security_id, condition, target_price, status
notification	notification_id → user_id, alert_no, anomaly_id, message, created_at, status; (user_id, anomaly_id) → notification_id
user_preference	user_id → price_alerts, anomaly_alerts, notification_mode

Step-by-step normalisation. A single flat "monitoring report" is used to show the process.

UNF (unnormalised form): repeating groups are shown in braces.

```
MONITOR(user_id, user_name, email,
  { watchlist_no, watchlist_name,
    { symbol, security_name, asset_class, status, company_name, sector_name,
      { exchange_name, country, timezone },
      { timestamp, timeframe, open, high, low, close, volume, trade_count, vwap,
        { anomaly_id, anomaly_type, anomaly_score, threshold,
          analysis_id, method, run_status, start_time, end_time } } } } )
```

1NF: the repeating groups are removed so that every attribute is atomic and each row is identified by a composite key.

```
R1(user_id, user_name, email, watchlist_no, watchlist_name, symbol, security_name,
   asset_class, status, company_name, sector_name, exchange_name, country, timezone,
   timestamp, timeframe, open, high, low, close, volume, trade_count, vwap,
   anomaly_id, anomaly_type, anomaly_score, threshold, analysis_id, method,
   run_status, start_time, end_time)
Key: (user_id, watchlist_no, symbol, exchange_name, timestamp, timeframe, anomaly_id)
```

2NF: partial dependencies on part of the composite key are removed.

```
user_id                      -> user_name, email
(user_id, watchlist_no)      -> watchlist_name
symbol                       -> security_name, asset_class, status, company_name,
sector_name
exchange_name                -> country, timezone
(symbol, timestamp, timeframe) -> open, high, low, close, volume, trade_count, vwap
anomaly_id                   -> anomaly_type, anomaly_score, threshold, analysis_id
analysis_id                  -> method, run_status, start_time, end_time
```

The result is app_user, watchlist, watchlist_security, security, exchange, exchange_security, price_bar, anomaly and analysis_run. In these relations every non-key attribute depends on the whole key.

3NF: transitive dependencies are removed. In `security`, the chain security_id → company → sector gives `security_id → company_id` and `company_id → sector_id → sector name`. This is decomposed into SECURITY(company_id), COMPANY(sector_id) and SECTOR(name). Likewise `method` and the run times stay in `analysis_run` and are reached from `anomaly` only through `analysis_id`. The user-owned relations alert, notification, user_preference and app_session already have no transitive dependencies.

BCNF check: in every relation listed above, the determinant of each functional dependency is a candidate key (for example, `symbol` and `security_id` are both candidate keys of `security`, and `email` and `user_id` are both candidate keys of `app_user`). The schema is therefore in BCNF, which also satisfies 3NF.

Deliberate design choices. (1) `anomaly` stores `anomaly_score` and `threshold` even though they derive from prices, because the rolling bound changes with every new bar and cannot be recomputed later for the same moment; keeping it supports traceability. (2) `anomaly.security_id` is kept as part of the composite foreign key to `price_bar`; the application ensures it matches the security of its analysis run. (3) A sector is a separate relation rather than a column of company so that sector names are stored once.

6. Implementation Details

6.1 Tech Stack & Database Connectivity

Layer	Technology	Purpose
DBMS	PostgreSQL 16.15 with TimescaleDB 2.30.0	Relational storage with time-series partitioning; Hypercore columnstore needs TimescaleDB 2.18 or newer (optional script)
Backend	Python 3.11.9, FastAPI, Uvicorn, Pydantic	REST API, validation, authentication, background workers
Database driver	psycopg 3 (`psycopg[binary]`) with `python-dotenv`	Connection through `database/connection.py` using DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD from `.env`
Analytics	pandas, NumPy, scikit-learn (listed dependency)	Rolling quantiles, returns and volume ratios
Market data	alpaca-py; Alpaca Markets, IEX feed	Historical bars over REST; live minute bars over an authenticated WebSocket
Frontend	React 18, Vite, Node.js 24.16.0, Chart.js through react-chartjs-2	Single-page dashboard with polling every 15 seconds
Testing	pytest, pytest-mock, httpx	Unit tests with mocked database and Alpaca; gated integration tests

Connectivity. Each operation opens a connection with `psycopg.connect(...)`, runs parameterised SQL (`%s` placeholders, never string formatting of user values), commits on success and rolls back on error. The API reads `.env` for credentials, and no endpoint returns connection strings or keys.

6.2 Key SQL Scripts & Operations

The listings below are taken from `sql/01_schema.sql` and the application code (formatting abridged). The last two queries are illustrative reporting queries for demonstration and are not part of the application code.

A. DDL: tables, constraints and indexes

`price_bar` uses a composite key that includes `timeframe` so that daily and minute bars never mix, and is converted to a TimescaleDB hypertable:

```
CREATE TABLE IF NOT EXISTS price_bar (
    security_id INTEGER NOT NULL
        REFERENCES security(security_id) ON DELETE CASCADE,
    timestamp TIMESTAMPTZ NOT NULL,
    timeframe VARCHAR(20) NOT NULL,
    open  NUMERIC(18,6) NOT NULL,  high NUMERIC(18,6) NOT NULL,
    low   NUMERIC(18,6) NOT NULL,  close NUMERIC(18,6) NOT NULL,
    volume BIGINT NOT NULL,
    trade_count INTEGER,  vwap NUMERIC(18,6),
    PRIMARY KEY (security_id, timestamp, timeframe),
    CONSTRAINT chk_price_bar_timeframe
        CHECK (timeframe IN ('day', 'minute'))
);
SELECT create_hypertable('price_bar', by_range('timestamp'),
                         if_not_exists => TRUE);
CREATE INDEX IF NOT EXISTS idx_price_bar_security_time
    ON price_bar (security_id, timestamp DESC);
```

`anomaly` has a composite foreign key to `price_bar` and a uniqueness rule that keeps re-runs idempotent:

```
CREATE TABLE IF NOT EXISTS anomaly (
    anomaly_id    SERIAL PRIMARY KEY,
    security_id   INTEGER NOT NULL,
    timestamp     TIMESTAMPTZ NOT NULL,
    timeframe     VARCHAR(20) NOT NULL,
    analysis_id   INTEGER NOT NULL
        REFERENCES analysis_run(analysis_id) ON DELETE CASCADE,
    anomaly_score NUMERIC(18,6) NOT NULL,
    threshold     NUMERIC(18,6) NOT NULL,
    anomaly_type  VARCHAR(100) NOT NULL,
    CONSTRAINT fk_anomaly_price_bar
        FOREIGN KEY (security_id, timestamp, timeframe)
        REFERENCES price_bar(security_id, timestamp, timeframe)
        ON DELETE CASCADE,
    CONSTRAINT uq_anomaly_run_bar_type UNIQUE
        (analysis_id, security_id, timestamp, timeframe, anomaly_type)
);
```

`notification` allows exactly one source and prevents duplicates:

```
CREATE TABLE IF NOT EXISTS notification (
    notification_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL
        REFERENCES app_user(user_id) ON DELETE CASCADE,
    alert_no INTEGER,
    anomaly_id INTEGER REFERENCES anomaly(anomaly_id) ON DELETE CASCADE,
    message TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) NOT NULL DEFAULT 'unread',
    CONSTRAINT fk_notification_alert FOREIGN KEY (user_id, alert_no)
        REFERENCES alert(user_id, alert_no) ON DELETE CASCADE,
    CONSTRAINT chk_notification_source CHECK (
        (alert_no IS NOT NULL AND anomaly_id IS NULL) OR
        (alert_no IS NULL AND anomaly_id IS NOT NULL)),
    CONSTRAINT uq_notification_user_anomaly UNIQUE (user_id, anomaly_id)
);
CREATE UNIQUE INDEX IF NOT EXISTS uq_notification_unread_alert
    ON notification (user_id, alert_no)
    WHERE alert_no IS NOT NULL AND status = 'unread';
```

Other integrity rules: `chk_alert_condition CHECK (condition IN ('above','below'))`, `chk_analysis_time CHECK (end_time >= start_time)` and `chk_notification_mode CHECK (notification_mode IN ('in_app','email'))`. Further indexes are `idx_anomaly_security_time` and `idx_notification_user_created`. `sql/05_price_bar_columnstore.sql` adds an optional columnstore policy that compresses chunks older than 14 days.

B. DML: insert, update, delete and select

```
-- Ingestion: re-loading the same bar updates it instead of failing
INSERT INTO price_bar (security_id, timestamp, timeframe, open, high, low,
                       close, volume, trade_count, vwap)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (security_id, timestamp, timeframe) DO UPDATE SET
    open = EXCLUDED.open, high = EXCLUDED.high, low = EXCLUDED.low,
    close = EXCLUDED.close, volume = EXCLUDED.volume,
    trade_count = EXCLUDED.trade_count, vwap = EXCLUDED.vwap;

-- Create a user price alert (alert_no = MAX + 1 within the user)
SELECT COALESCE(MAX(alert_no), 0) + 1 FROM alert WHERE user_id = %s;
INSERT INTO alert(user_id, alert_no, security_id, condition,
                  target_price, status)
VALUES (%s, %s, %s, %s, %s, 'active');

-- Notify a watcher of an anomaly; a duplicate is silently skipped
INSERT INTO notification (user_id, alert_no, anomaly_id, message,
                          created_at, status)
VALUES (%s, NULL, %s, %s, CURRENT_TIMESTAMP, 'unread')
ON CONFLICT (user_id, anomaly_id) DO NOTHING
RETURNING notification_id;

-- Mark a notification as read (restricted to its owner)
UPDATE notification SET status = 'read'
WHERE user_id = %s AND notification_id = %s
RETURNING notification_id, status;

-- Save preferences: insert the row or update it if it exists
INSERT INTO user_preference(user_id, price_alerts, anomaly_alerts,
                            notification_mode)
VALUES (%s, %s, %s, %s)
ON CONFLICT (user_id) DO UPDATE SET
    price_alerts = EXCLUDED.price_alerts,
    anomaly_alerts = EXCLUDED.anomaly_alerts,
    notification_mode = EXCLUDED.notification_mode;

-- Delete a watchlist; its watchlist_security rows are removed by CASCADE
DELETE FROM watchlist WHERE user_id = %s AND watchlist_no = %s
RETURNING name;
```

C. Queries: joins, aggregates, subqueries and window functions

```
-- 1. Price history for a symbol and timeframe (JOIN)
SELECT pb.timestamp, pb.open, pb.high, pb.low, pb.close, pb.volume,
       pb.trade_count, pb.vwap
FROM price_bar pb JOIN security s ON s.security_id = pb.security_id
WHERE s.symbol = %s AND pb.timeframe = %s
ORDER BY pb.timestamp DESC LIMIT %s;

-- 2. Anomaly summary per security (aggregate with FILTER)
SELECT COUNT(*) FILTER (WHERE anomaly_type LIKE '%return%') AS price,
       COUNT(*) FILTER (WHERE anomaly_type LIKE '%volume_spike%') AS spikes,
       COUNT(*) FILTER (WHERE anomaly_type LIKE '%volume_decline%') AS declines,
       COUNT(*) AS total
FROM anomaly WHERE security_id = %s;

-- 3. Users who watch a security (multi-table join)
SELECT DISTINCT w.user_id
FROM watchlist w JOIN watchlist_security ws
  ON ws.user_id = w.user_id AND ws.watchlist_no = w.watchlist_no
WHERE ws.security_id = %s;

-- 4. Active alerts with their security (join)
SELECT a.user_id, a.alert_no, a.security_id, s.symbol,
       a.condition, a.target_price
FROM alert a JOIN security s ON s.security_id = a.security_id
WHERE a.status = 'active';

-- 5. Baseline history: the prior N minute bars, current bar excluded
SELECT timestamp, close, volume FROM price_bar
WHERE security_id = %s AND timeframe = 'minute' AND timestamp < %s
ORDER BY timestamp DESC LIMIT %s;
```

Query 5 feeds the rolling Tukey computation: Q1 and Q3 are the 25th and 75th percentiles of the prior returns, and the bounds are Q1 − 1.5 × IQR and Q3 + 1.5 × IQR. A return or volume ratio outside these bounds is stored as a `realtime_positive_return_spike`, `realtime_negative_return_spike`, `realtime_volume_spike` or `realtime_volume_decline` anomaly.

```
-- 6. Illustrative reporting query: GROUP BY, HAVING and joins
SELECT s.symbol, a.anomaly_type, COUNT(*) AS occurrences,
       ROUND(AVG(a.anomaly_score), 4) AS avg_score
FROM anomaly a
JOIN security s ON s.security_id = a.security_id
JOIN analysis_run r ON r.analysis_id = a.analysis_id
GROUP BY s.symbol, a.anomaly_type
HAVING COUNT(*) > 1
ORDER BY occurrences DESC;

-- 7. Illustrative subquery: users who watch a security that has anomalies
SELECT u.user_id, u.email FROM app_user u
WHERE u.user_id IN (
    SELECT ws.user_id FROM watchlist_security ws
    WHERE ws.security_id IN (SELECT DISTINCT security_id FROM anomaly));
```

D. Transactions and concurrency control

Every write path commits only after all its statements succeed and rolls back otherwise (for example `price_ingestion.py` and `metadata_ingestion.py`). The migration `sql/04_notification_deduplication.sql` runs inside `BEGIN ... COMMIT` and uses a window function to keep only the earliest duplicate unread notification before creating the unique index:

```
WITH ranked AS (
    SELECT notification_id,
           ROW_NUMBER() OVER (PARTITION BY user_id, alert_no
                              ORDER BY created_at, notification_id) AS duplicate_rank
    FROM notification
    WHERE alert_no IS NOT NULL AND status = 'unread')
DELETE FROM notification n USING ranked r
WHERE n.notification_id = r.notification_id AND r.duplicate_rank > 1;
```

The automation owner is chosen with `SELECT pg_try_advisory_lock(...)`, so only one API worker runs background jobs. The repository defines no SQL views or stored procedures; all logic is in the Python layer, with integrity enforced by the schema.

6.3 Core Application Features (At least 5)

Each feature below is implemented end to end, from the dashboard, through the API, to the database.

#	Feature	Dashboard action	Backend (API and module)	Database operations
1	Account registration and sign-in	Login page: register or sign in	`POST /auth/register`, `/auth/login`, `/auth/logout`, `/auth/me`; scrypt hashing in `auth.py`	INSERT `app_user` and `user_preference`; INSERT `app_session` (token hash); SELECT to verify; 409 on duplicate email
2	Market data ingestion	Run from CLI or by automation	`ingestion/price_ingestion.py`, `metadata_ingestion.py`	Upsert `price_bar`; insert `sector`, `company`, `security`, `exchange`, `exchange_security`
3	Historical anomaly analysis	"Run historical analysis"	`POST /analysis/run/{symbol}` → `analysis/run_analysis.py`	INSERT `analysis_run`; INSERT `anomaly` (ON CONFLICT DO NOTHING); UPDATE run status
4	Real-time detection pipeline	Live stream, or "Process latest bar"	`POST /pipeline/process/{symbol}`; `pipeline.process_new_bar`	INSERT `price_bar`; SELECT prior 21 bars; INSERT `anomaly`; INSERT `notification`
5	Charts and adaptive thresholds	Monitoring tab: price and volume charts, markers, recent anomalies	`GET /stocks`, `/prices`, `/anomalies`, `/summary`, `/thresholds`	SELECT with joins; aggregate counts; rolling IQR computed from the last bars
6	Watchlists	Watchlists tab: create, add or remove stocks, delete	`/users/{id}/watchlists` (GET, POST, DELETE)	INSERT, DELETE on `watchlist`, `watchlist_security` (CASCADE)
7	Price alerts	Price Alerts tab: create, evaluate, delete	`/users/{id}/alerts`, `/alerts/check`; `alerts/alert_checker.py`	INSERT `alert`; SELECT latest `price_bar`; INSERT `notification` (one unread per alert)
8	Notifications and preferences	Notifications and Preferences tabs	`GET /notifications`, `PATCH .../read`, `GET/PUT /preferences`	UPDATE `notification`; upsert `user_preference`
9	My Data Queries	Guided query builder with read-only SQL preview	`api/user_query_builder.py`	SELECT only on the user's own rows; allowlisted columns; 100-row limit; 3-second timeout

7. Testing, Screenshots & Results

7.1 Input Validation & Exception Handling

Validation happens in three layers: the browser and Pydantic models reject malformed input, explicit checks in the API return clear errors, and the database constraints act as the final safeguard.

Case	Where it is caught	Handling and result
Duplicate e-mail on registration	UNIQUE constraint on `app_user.email` (UniqueViolation)	HTTP 409 with a readable message
Blank name or invalid e-mail; name longer than 200 characters	Regular-expression and length checks in `/auth/register`	HTTP 422
Password shorter than 12 characters	Pydantic `Field(min_length=12)`	HTTP 422
Wrong password or unknown user	`verify_password` (constant-time compare)	HTTP 401 with a generic message, so the response does not reveal which part was wrong
Missing, expired or another user's session	`get_current_user`, `require_matching_user`	HTTP 401, or 404 for another user's records
Unknown stock symbol	Lookup in `security` before use	HTTP 404 with JSON `{"error": ...}`
Invalid timeframe, alert condition or notification mode	API checks and database CHECK constraints	HTTP 422
Target price zero, negative, NaN or infinite	`Field(gt=0, allow_inf_nan=False)`	HTTP 422
Empty or over-long watchlist name	`Field(min_length=1, max_length=100)` and blank check	HTTP 422
Adding the same stock to a watchlist twice	`ON CONFLICT DO NOTHING` on `watchlist_security`	Idempotent; no error and no duplicate row
Duplicate notification for the same anomaly or alert	`UNIQUE(user_id, anomaly_id)` and the partial unique index; `ON CONFLICT DO NOTHING`	Second insert is skipped, also under concurrent workers
Notification with both or neither source	CHECK `chk_notification_source`	Database rejects the row
Anomaly pointing to a missing price bar	Composite foreign key to `price_bar`	Database rejects the row
Re-ingesting an existing bar	Upsert on the `price_bar` primary key	Row updated; no duplicate
Too little history for a baseline, or a flat series (zero IQR)	`realtime_detector.py`, `rolling_iqr_bounds`	No event is produced; no NaN or infinite bound; not treated as an error
Symbol absent from Alpaca or from the database	Ingestion checks	Symbol is skipped with a message; the batch continues
Database error during a write	`try / except` with `rollback()`; `/health` endpoint	Transaction rolled back; `/health` reports unhealthy if the database is unreachable
SQL injection	Parameterised queries; the query builder accepts no SQL text	Values are bound separately; table, column and action names come from a server allowlist

7.2 Test Cases & Execution Results

The test command was python -m pytest -m "not integration" -v. The run completed with 70 passed, 17 failed, and 9 deselected; two warnings were reported. All 17 failures occurred in tests/test_api.py.

Several endpoint tests received HTTP 401 where the tests expected success or validation errors, while others failed because the returned response did not contain keys expected by the tests (for example, stock, message, notification_mode, alerts_checked, or symbol). The analysis endpoint tests also received an authentication response instead of the expected result. These failures show that the API tests need to be brought into line with the current authentication and response behavior; they are not evidence that those endpoints passed. The team should include this outcome as a limitation and rerun the suite after the tests are updated.

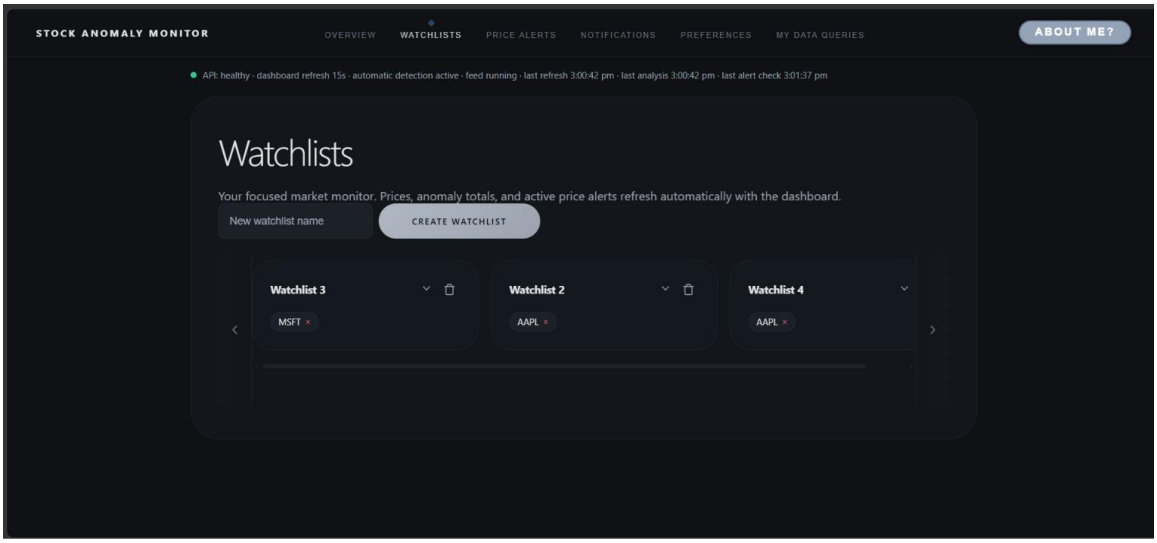
Test ID	Feature / Description	Expected Outcome	Actual Outcome	Status
TC01	Rolling baseline excludes the current observation	Current bar does not move its own baseline	As expected	PASS
TC02	Large positive return (`test_combined_analysis`)	Flagged as a positive return spike	Flagged	PASS
TC03	Normal movement	Not flagged	Not flagged	PASS
TC04	Flat history with zero variability	No NaN or infinite bound, no false flag	As expected	PASS
TC05	Real-time price and volume spike with enough history	Anomalies returned	Returned	PASS
TC06	Real-time bar with insufficient history	No events and no error	As expected	PASS
TC07	Pipeline processes a new bar	Bar stored, anomaly stored, users notified, alerts checked	As expected	PASS
TC08	Price alert condition met	Alert triggers and a notification is created	As expected	PASS
TC09	Price alert condition not met	No notification	As expected	PASS
TC10	Alert with an unread notification already present	No second notification	As expected	PASS
TC11	Concurrent duplicate alert insert	Handled safely, no duplicate	As expected	PASS
TC12	User has price alerts disabled	Alert skipped	As expected	PASS
TC13	Anomaly for a watched stock	Only watchers with anomaly alerts enabled are notified	As expected	PASS
TC14	Same anomaly notified twice	Second notification skipped	As expected	PASS
TC15	Price ingestion, unknown security	Symbol skipped, others inserted	As expected	PASS
TC16	Metadata ingestion error	Transaction rolled back	As expected	PASS
TC17	`GET /stocks/{symbol}/prices` with invalid timeframe	Request rejected with 422	As expected	PASS
TC18	`GET /health` with database unreachable	Reports unhealthy	As expected	PASS
TC19	Historical analysis for an unknown symbol	Raises an error	As expected	PASS
TC20	Manual: sign in with valid credentials	Redirected to the dashboard	As expected	PASS
TC21	Manual: register with an existing e-mail	Rejected with HTTP 409 and a message	As expected	PASS
TC22	Manual: create an alert with target price 0	Rejected with HTTP 422	As expected	PASS
TC23	Manual: "Process latest bar" on a watched stock	Anomaly and notification appear after refresh	As expected	PASS

7.3 Application Screenshots & Database Evidence

The figures below place each application view beside its corresponding database query result. Together, they show how the system presents and stores market data, anomalies and user-specific records.

Figure 7.1 Watchlist records for user ID 2

The dashboard shows three watchlists. The pgAdmin query below returns the corresponding three rows for user_id = 2.



Application screenshot: Watchlists

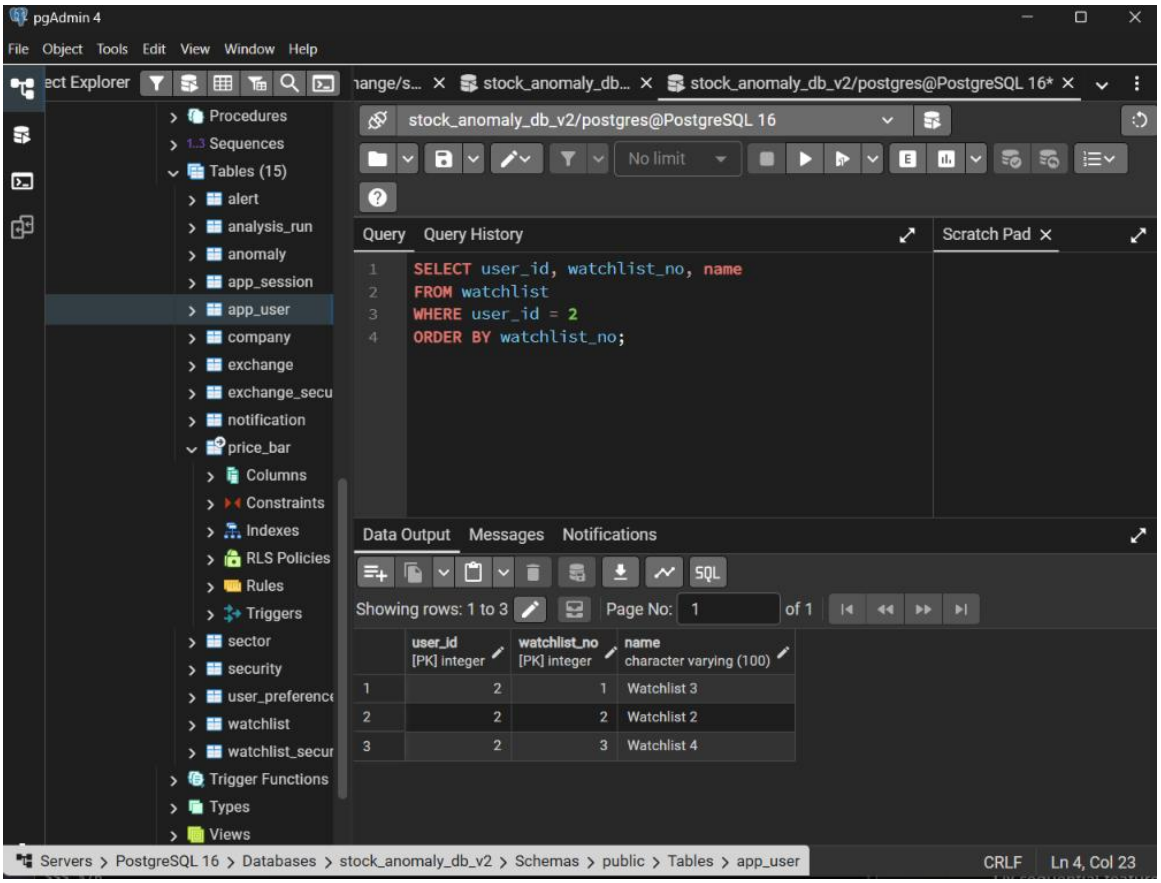
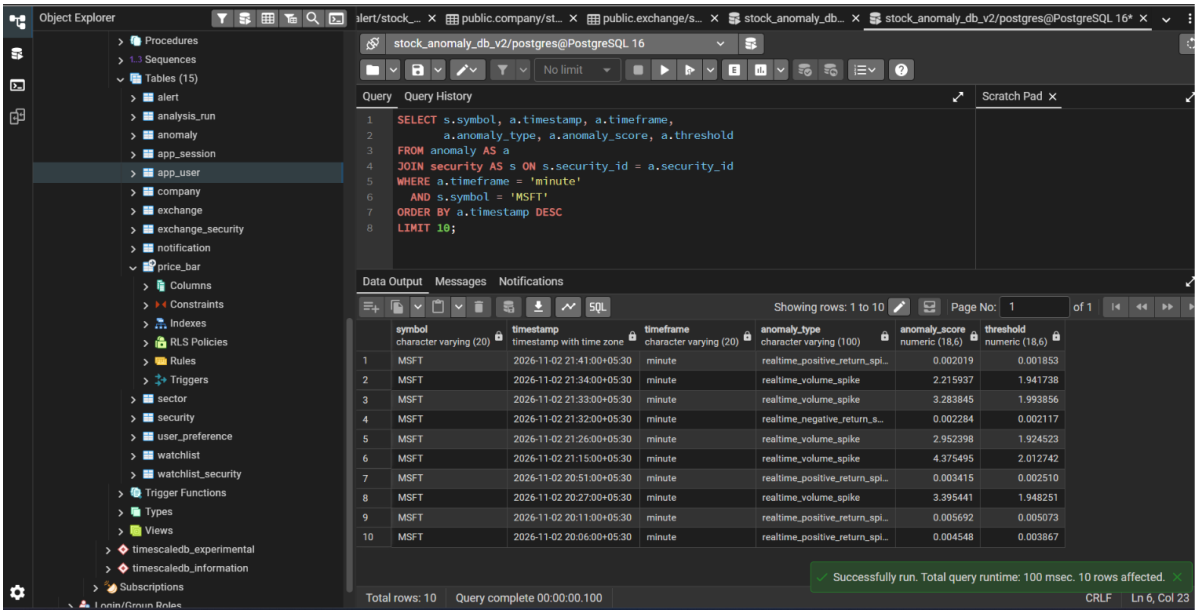


Figure 7.2 Intraday MSFT anomaly evidence

The pgAdmin result confirms minute-level anomaly records for MSFT, and the intraday chart displays anomaly markers for plotted observations. The latest database anomaly shown is timestamped 2 November 2026 at 21:41 (+05:30), while the chart spans into 3 November. Therefore, these screenshots demonstrate that anomaly records are stored and visualized, but do not establish a one-to-one match for the newest anomaly and chart bar.



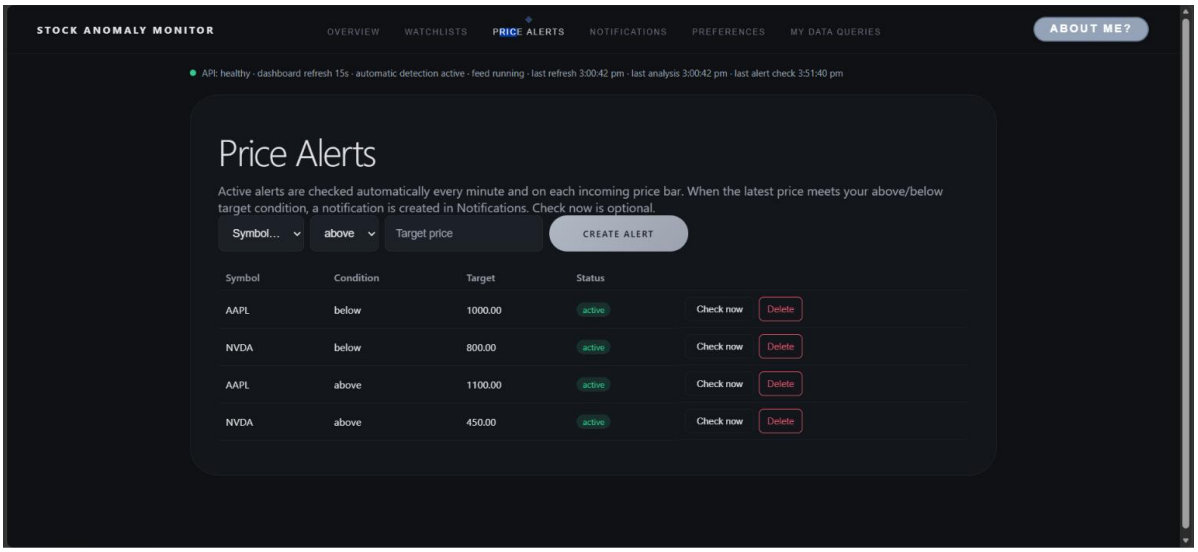
MSFT intraday chart after reprocessing



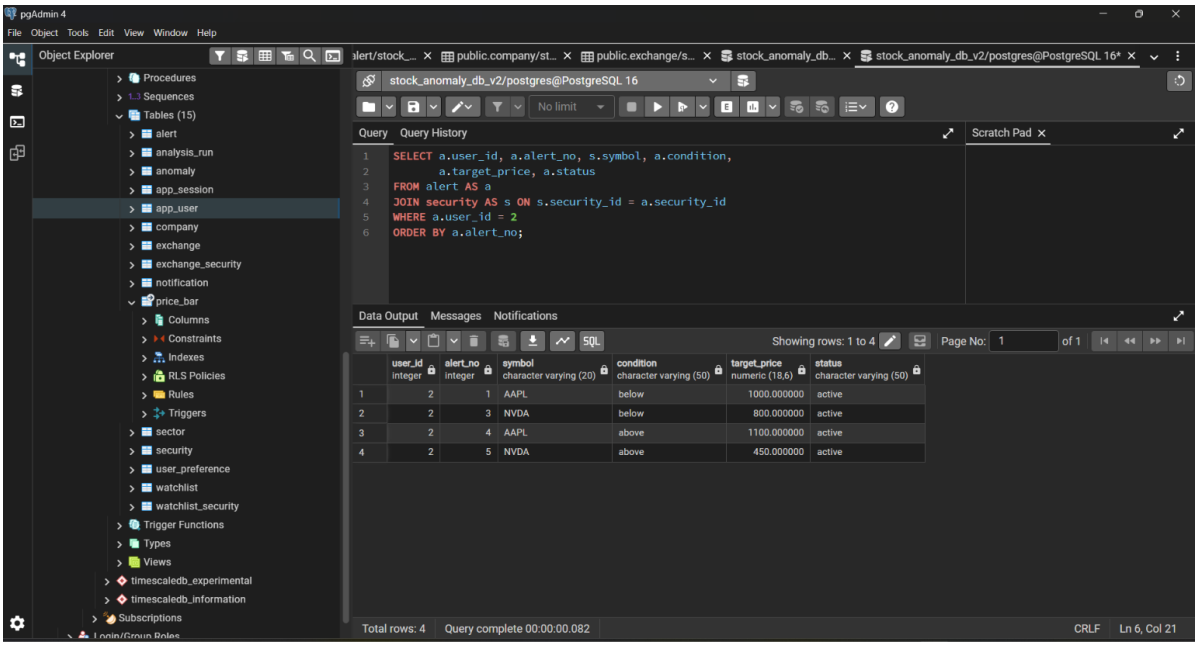
Matching pgAdmin query filtered to MSFT minute anomalies

Figure 7.3 Price alert records for user ID 2

The dashboard and pgAdmin query show the same four active alerts for user_id = 2: AAPL below 1000, NVDA below 800, AAPL above 1100 and NVDA above 450. The query joins alert to security to display each symbol. This evidence confirms that the alerts shown in the application are stored in the database; it does not by itself demonstrate a price condition being triggered.



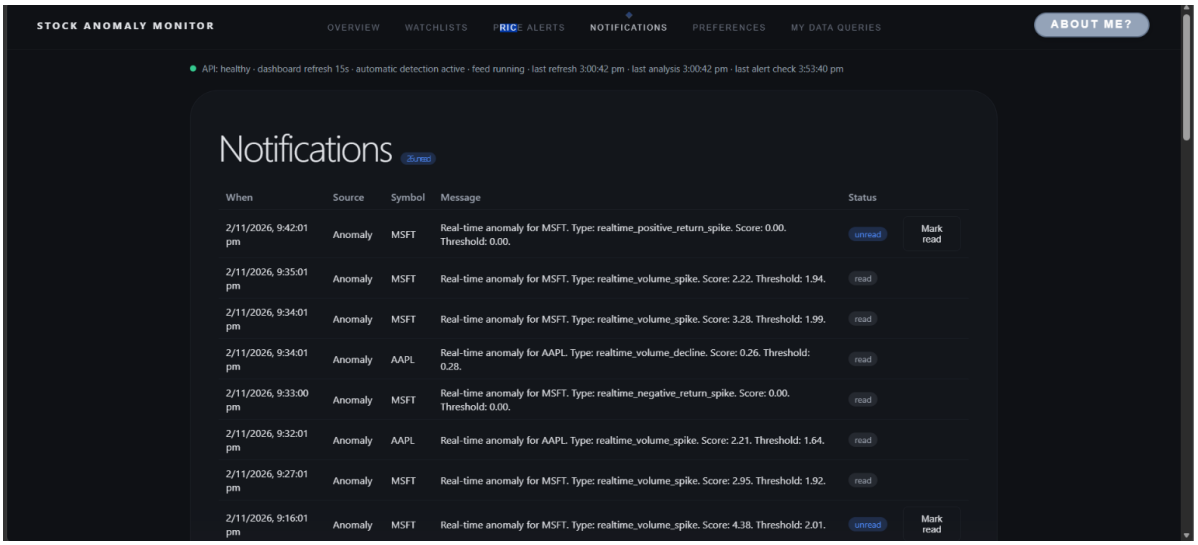
Price Alerts page showing the four active alerts



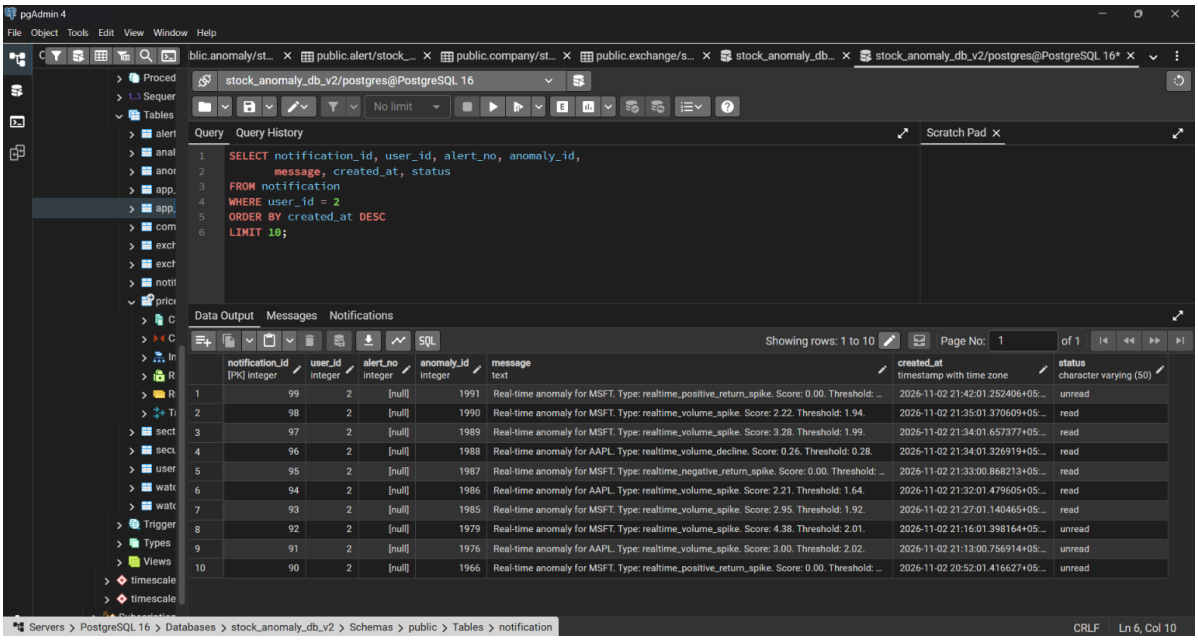
pgAdmin query result for alert rows belonging to user ID 2

Figure 7.4 Notification records for user ID 2

The Notifications page and the query results show the same recent anomaly notifications. The newest visible entry is an unread MSFT positive return spike at approximately 9:42 p.m. on 2 November 2026; the database row records it at 21:42 with the +05:30 timezone. Older visible entries also correspond by symbol, message and read status. This demonstrates that anomaly notifications are persisted and retrieved for the signed-in user.



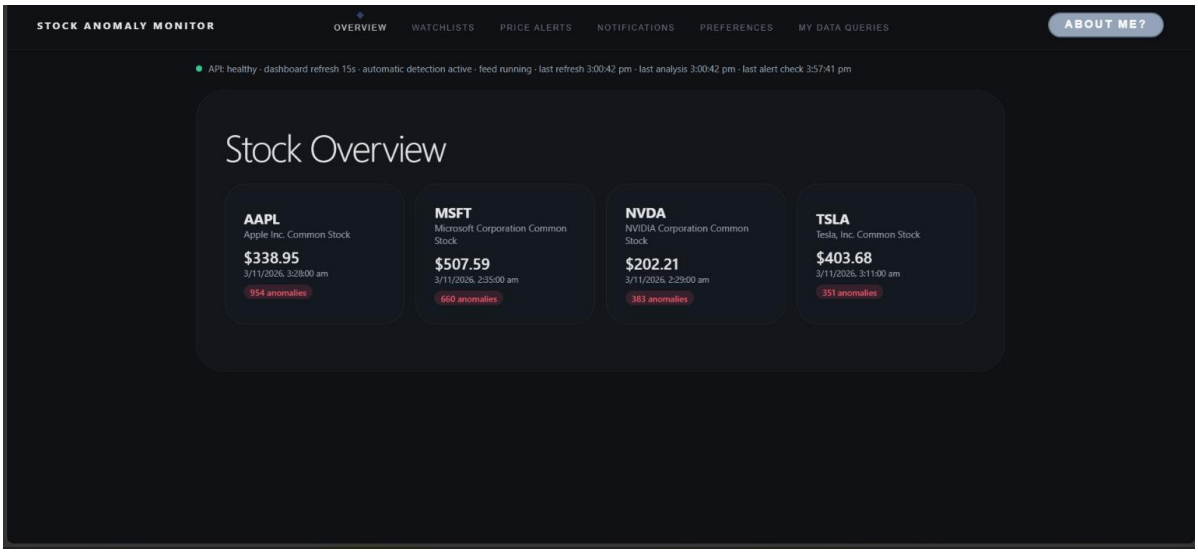
Notifications page showing recent anomaly messages and read status



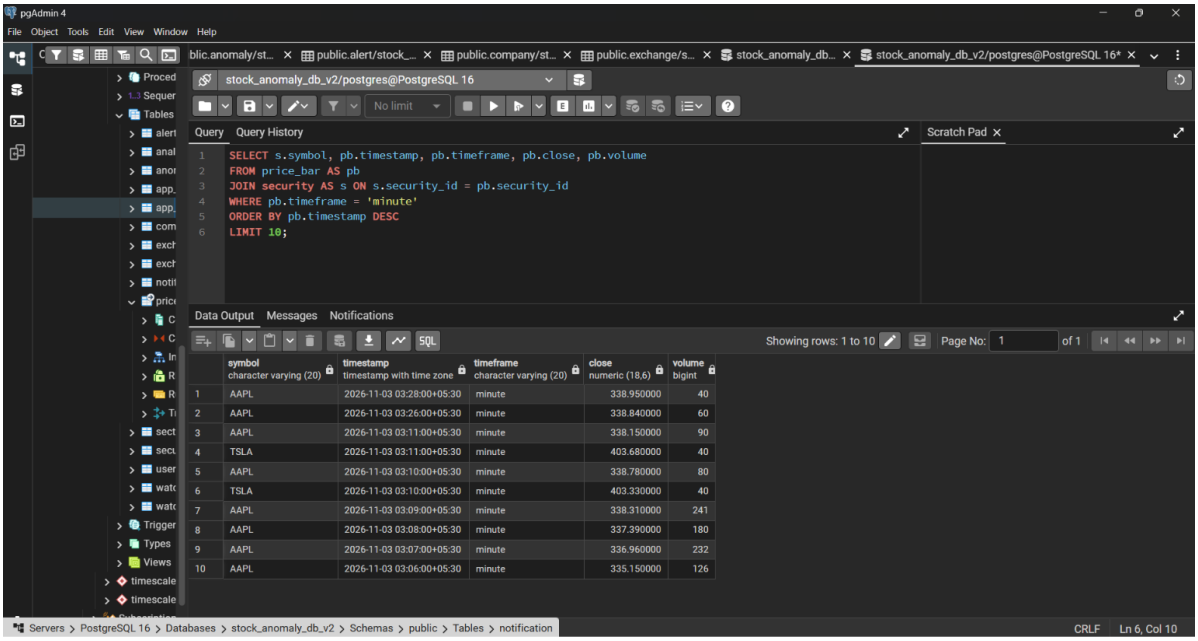
pgAdmin result listing the ten most recent notifications for user ID 2

Figure 7.5 Overview price data and database records

The Overview cards display current prices and their latest bar times. The pgAdmin result independently shows the corresponding latest minute bars, including AAPL at 338.95 at 03:28 (+05:30) and TSLA at 403.68 at 03:11 (+05:30), matching the values and times shown in the dashboard.



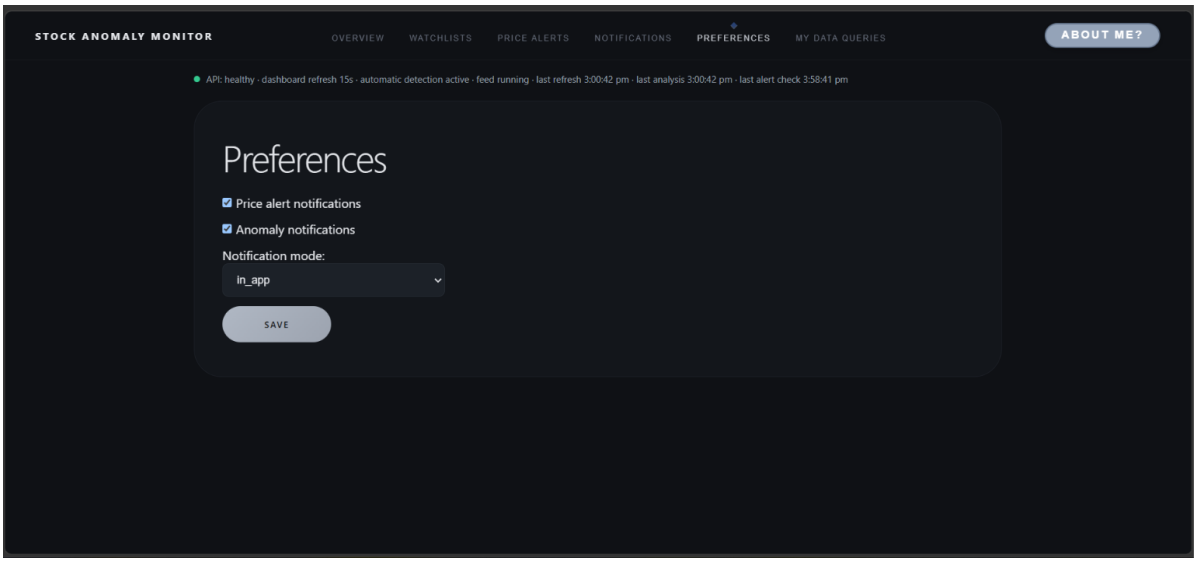
Stock Overview dashboard



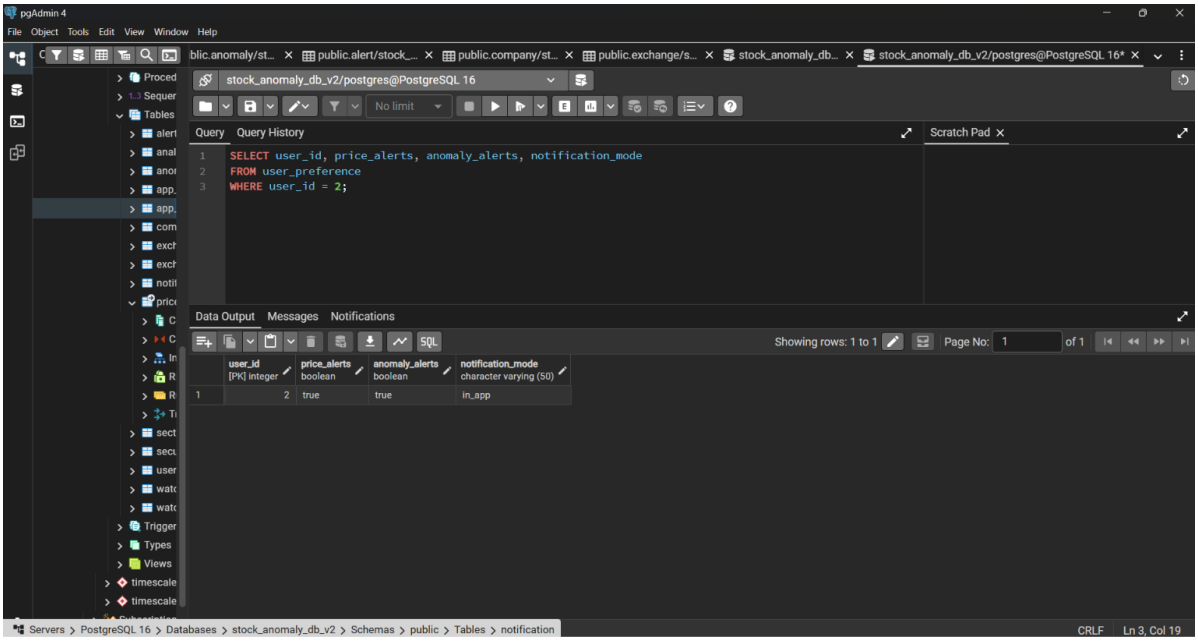
pgAdmin latest minute bars joined with security symbols

Figure 7.6 Saved notification preferences

The Preferences page shows price alerts and anomaly notifications enabled, with the notification mode set to in_app. The user_preference row for user_id = 2 contains price_alerts = true, anomaly_alerts = true and notification_mode = in_app, confirming that the displayed settings are stored in PostgreSQL.



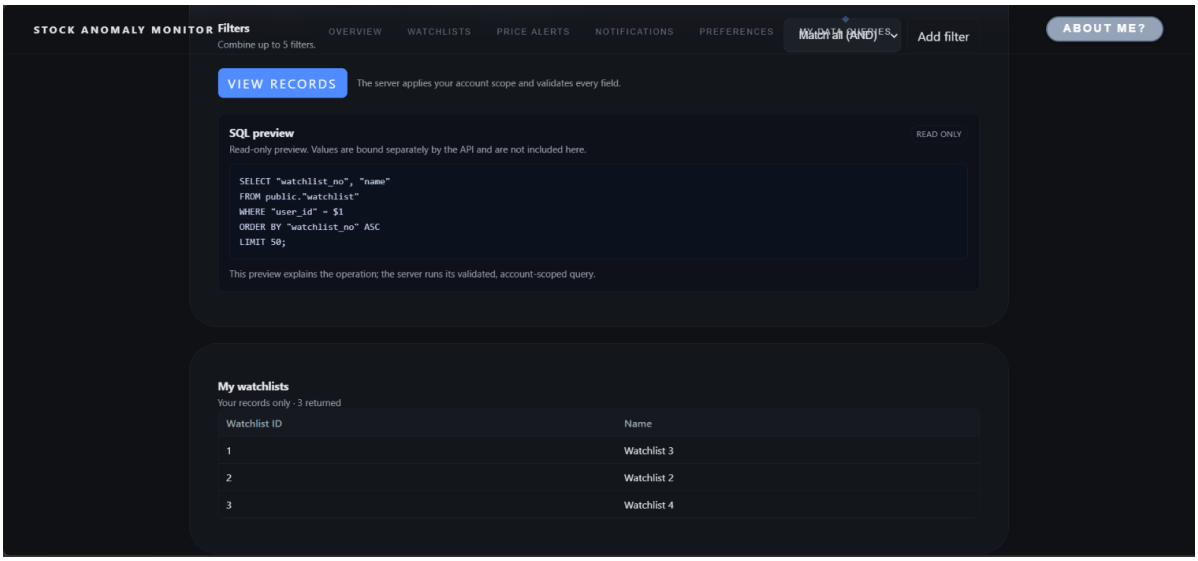
Preferences page



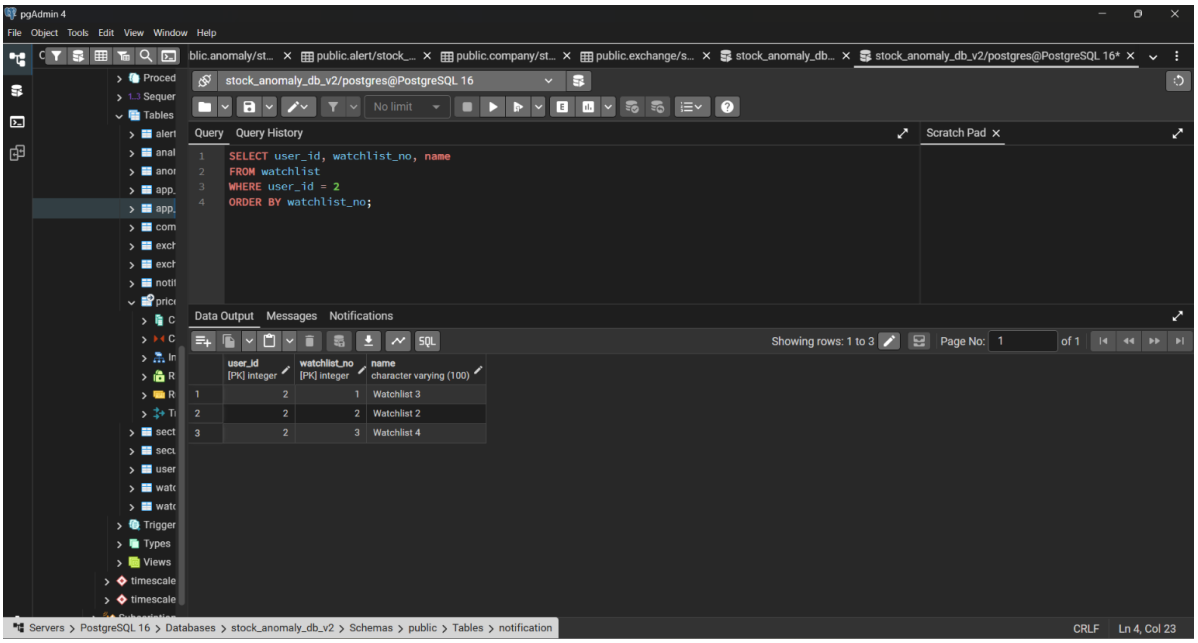
pgAdmin result for user ID 2 preferences

Figure 7.7 Account-scoped My Data Queries result

The My Data Queries page displays a read-only SQL preview and returns three watchlists. The preview scopes the request with a bound user parameter, and the returned IDs and names match the pgAdmin rows for user_id = 2. This demonstrates the query interface and its database result for a read operation.



My Data Queries page showing the SQL preview and watchlist rows



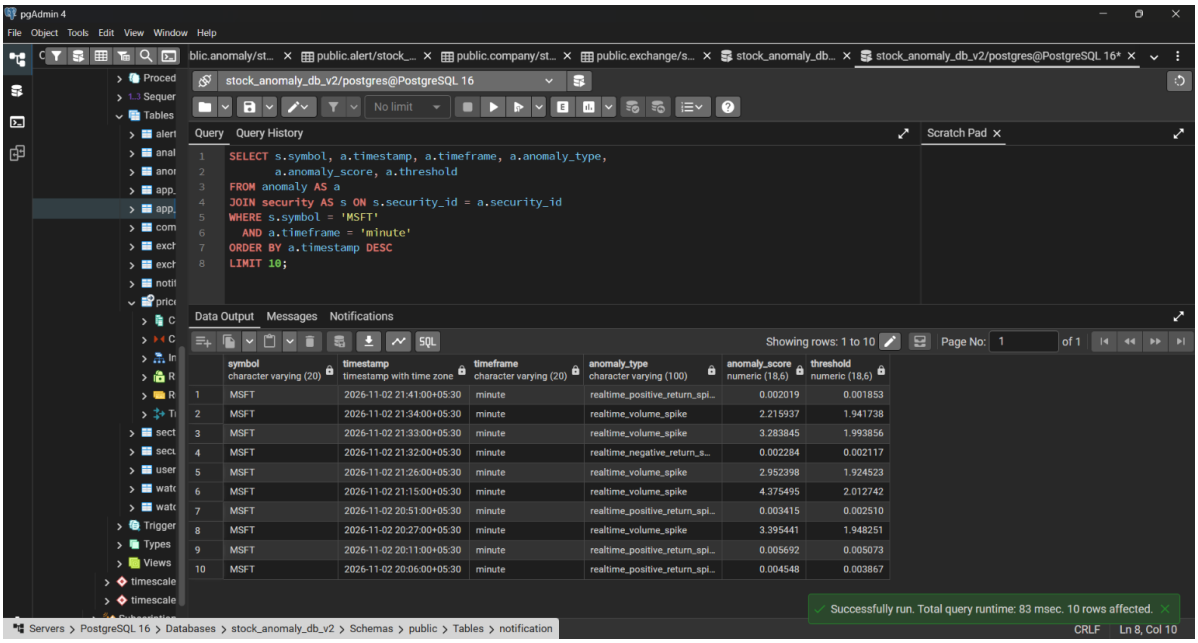
pgAdmin watchlist rows for user ID 2

Figure 7.8 MSFT intraday anomaly and adaptive threshold evidence

The MSFT intraday monitor displays rolling return and volume limits calculated from the prior 20 bars, along with anomaly markers on the price chart. The pgAdmin query shows minute-level anomaly scores and their stored thresholds for MSFT. Together these demonstrate adaptive threshold processing and anomaly display; the screenshots do not prove that the newest database row maps to a particular chart marker.



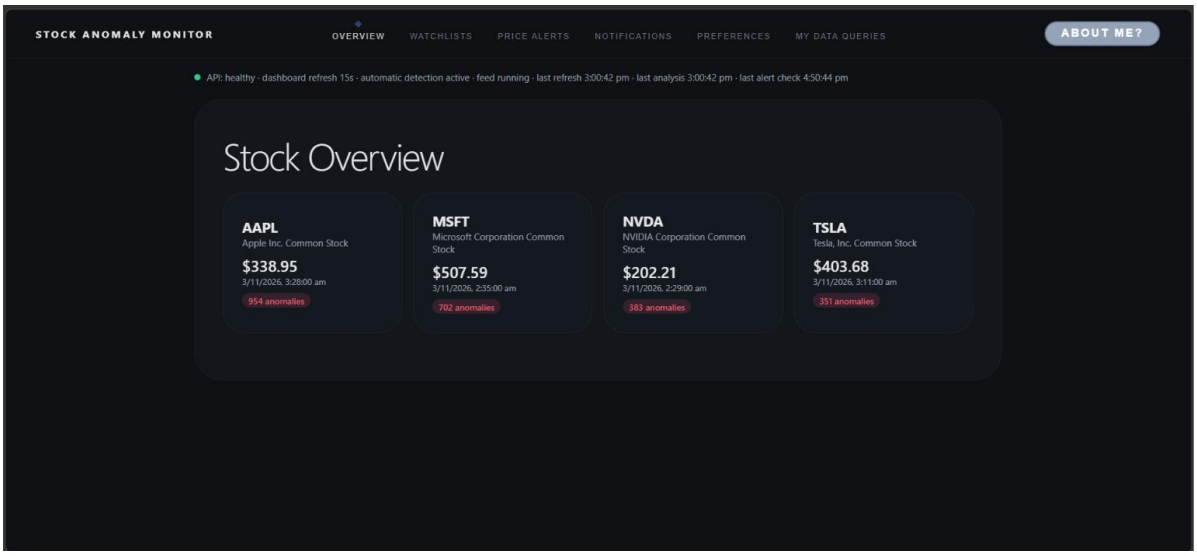
MSFT intraday chart showing adaptive limits and anomaly markers



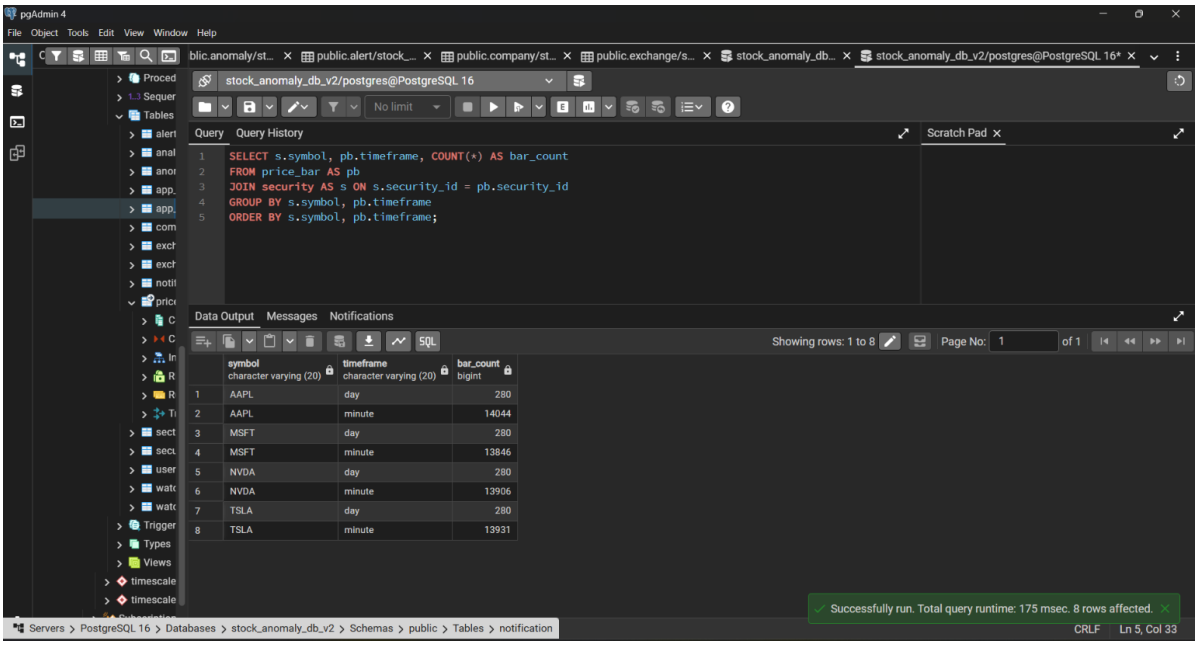
pgAdmin results showing recent MSFT minute anomaly scores and thresholds

Figure 7.9 Ingested market data by symbol and timeframe

The Overview page shows the latest stock prices and bar times for the monitored symbols. The grouped database query confirms that daily and minute price bars are stored for AAPL, MSFT, NVDA and TSLA. The counts provide a database-side summary of the ingested market data.



Stock Overview showing monitored symbols, latest prices and timestamps



pgAdmin grouped price-bar counts by symbol and timeframe

8. Conclusion, Limitations & Future Enhancements

The project delivers a working end-to-end system in which a relational, time-series-aware database is central to both correctness and traceability. Historical and live market data are loaded into PostgreSQL with TimescaleDB; per-stock baselines are learned from rolling distributions; anomalies are detected with transparent Tukey fences and stored with the run and price bar that produced them; and users

manage watchlists, alerts, preferences and notifications through a secured REST API and a React dashboard. Composite keys, weak entities, CHECK constraints and partial unique indexes keep the data consistent, and the schema is normalised to BCNF.

Limitations

- **IEX feed, not consolidated SIP data.** Alpaca's free tier provides only the IEX feed, so this is an IEX-feed prototype; `config.py` sets `DATA_FEED = "iex"`.
- **No time-of-day normalisation.** The intraday baseline is a rolling window of the latest minute bars and is not compared with the same time on earlier days.
- **Dashboard updates by polling.** The browser polls every 15 seconds; the backend-to-Alpaca link is a genuine WebSocket.
- **E-mail notifications are not implemented.** `notification_mode = 'email'` is stored only; delivery is in-app.
- **Sector data is a manual mapping** because Alpaca does not provide sector classification.
- **Legacy accounts need credentials.** Accounts created before password sign-in have a NULL password hash.
- **Small symbol universe.** Only four symbols are configured, and re-running historical analysis creates a new `analysis_run` each time by design.
- **Test suite needs updating.** The 17 API tests listed in Section 7.2 must be adapted to the authenticated session.
- **Scale.** The design targets a single database server; no partitioning by user, replication or load testing has been done.

Future enhancements

- Time-of-day volume normalisation so each minute is compared with the same time on previous sessions.
- Real e-mail or push delivery for the `email` notification mode, and browser WebSocket push instead of polling.
- A consolidated (SIP) data feed and a larger, user-selectable symbol universe.
- Additional detectors (for example Isolation Forest or seasonal models) recorded as separate methods in `analysis_run` for comparison.
- Database views and stored procedures or triggers for reporting and alert evaluation, and role-based access control.
- Load testing and TimescaleDB continuous aggregates for faster dashboard queries.

9. References

[1] Alpaca Markets, "Alpaca Market Data API Documentation," [Online]. Available: https://docs.alpaca.markets

[2] Timescale, "TimescaleDB Documentation," [Online]. Available: https://docs.timescale.com

[3] The PostgreSQL Global Development Group, "PostgreSQL Documentation," [Online]. Available: https://www.postgresql.org/docs/

[4] psycopg Team, "Psycopg 3 Documentation," [Online]. Available: https://www.psycopg.org/psycopg3/docs/

[5] S. Ramírez, "FastAPI Documentation," [Online]. Available: https://fastapi.tiangolo.com

[6] Meta Open Source, "React Documentation," [Online]. Available: https://react.dev

[7] J. W. Tukey, Exploratory Data Analysis. Reading, MA, USA: Addison-Wesley, 1977.

[8] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, pp. 1-58, 2009.

[9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413-422.

[10] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[11] TradingView, [Online]. Available: https://www.tradingview.com; Yahoo Finance, [Online]. Available: https://finance.yahoo.com; Bloomberg, "Bloomberg Terminal," [Online]. Available: https://www.bloomberg.com/professional/products/bloomberg-terminal/